Strategic Blueprint for a Semester-Long Capstone: Architecting a Federated Social Networking Platform with Decentralized Identity

Executive Summary

The digital landscape is currently witnessing a paradigm shift from centralized, siloed social networking architectures toward decentralized, federated ecosystems known collectively as the "Fediverse." This transition is driven by increasing concerns regarding data privacy, algorithmic opacity, and the fragility of centralized identity providers. This report outlines a comprehensive execution plan for a semester-long software engineering capstone project designed to build a **Federated Social Networking Platform**. The proposed system integrates the **ActivityPub** protocol for interoperability with existing networks (e.g., Mastodon, PixelFed) and **Decentralized Identifiers (DIDs)** to ensure user data autonomy.

The project is structured according to **Agile methodology**, tailored for a cross-functional team comprising Frontend, Backend, DevOps, and Testing engineers. The development lifecycle is segmented into five distinct **Epics**, each decomposing into five client-centric **User Stories**, which are further broken down into granular technical implementation tasks. This structure ensures a steady velocity of delivery, risk mitigation, and the progressive layering of complexity—from basic identity management to asynchronous federation and cryptographic trust mechanisms.

To ensure accessibility and pedagogical value, the report recommends a **Free and Open-Source Software (FOSS)** technology stack. This selection prioritizes widely adopted standards (TypeScript, React, PostgreSQL) alongside specialized libraries (Fedify) that abstract protocol complexity while allowing deep architectural engagement. The analysis herein draws upon extensive technical documentation, W3C specifications, and industry best practices to provide a rigorous roadmap for student success.

---

1. Architectural Vision and Pedagogical Context

1.1 The Imperative of Decentralization

Contemporary social media operates on a client-server model where the service provider acts as the custodian of identity, content, and the social graph. This centralization creates a single point of failure and control. The **Fediverse**, underpinned by the **ActivityPub** protocol [1], offers

an alternative where independent servers ("instances") communicate via a standard API, allowing users on one server to interact seamlessly with users on another. This model mirrors the interoperability of email protocols (SMTP/IMAP), where a Gmail user can message an Outlook user without friction.[3]

However, standard ActivityPub implementations typically bind a user's identity to the server's domain (e.g., @user@instance.com). If the instance goes offline or the administrator bans the account, the user loses their digital identity and social connections. This project introduces **Decentralized Identifiers (DIDs)** [4] to decouple identity from infrastructure. By leveraging W3C DID standards, the platform ensures that the user's identity is cryptographically rooted, theoretically enabling portability across different hosting providers without loss of reputation or social graph.

1.2 Technology Stack Selection

The selection of the technology stack is critical for a semester-long project. It must balance industry relevance, learning curve, and cost efficiency. The following stack is proposed based on its robustness, open-source nature, and suitability for the specific challenges of ActivityPub and DID implementation.

Component	Technology	Rationale and Technical Justification
Language	**TypeScript** (Node.js)	ActivityStreams 2.0, the data format of ActivityPub, is JSON-LD based. TypeScript provides essential type safety for complex nested objects, reducing runtime errors associated with dynamic payloads.[6]
Backend Framework	**Fedify**	Unlike generic frameworks like Express, Fedify is purpose-built for ActivityPub. It handles Actor dispatching, WebFinger resolution, and HTTP Signatures out-of-the-box, allowing the team to focus on application logic rather than protocol plumbing.[8]
Frontend Library	**React**	The component-based architecture of React is ideal for rendering recursive social threads and managing the

		complex state of real-time feeds. Its ecosystem offers robust tools for rich text editing and media handling.[10]
Database	**PostgreSQL**	ActivityPub data is semi-structured (JSON-LD), yet user relationships are relational. PostgreSQL's JSONB support offers the best of both worlds, allowing schema flexibility for federation while maintaining ACID compliance for local data.[12]
Job Queue	**Redis** (BullMQ)	Federation requires sending HTTP requests to thousands of remote servers. This must be handled asynchronously to prevent blocking the main thread. Redis provides the necessary speed and persistence for these job queues.[10]
Identity Protocol	**did:web**	While did:key is simpler, did:web allows the DID to be resolved via standard DNS and HTTP, making it easier to integrate with the existing web infrastructure of ActivityPub without requiring a blockchain.[14]
Hosting	**Railway / Render**	Both platforms offer generous free tiers that support Node.js, PostgreSQL, and Redis. They integrate directly with GitHub for automated CI/CD pipelines, mimicking a professional DevOps environment.[16]

---

2. Agile Execution Strategy

The semester is divided into five Epics, each spanning roughly 3 weeks. The Agile workflow will necessitate weekly sprints, with the team rotating the "Scrum Master" role to ensure all members gain leadership experience.

Team Roles:

- **Frontend Engineer (FE):** Responsible for the User Interface (UI), User Experience (UX), and client-side state management.
- **Backend Engineer (BE):** Responsible for API development, database schema design, protocol implementation, and business logic.
- **DevOps Engineer (DO):** Responsible for CI/CD pipelines, infrastructure management, security configuration, and monitoring.
- **Testing Engineer (QA):** Responsible for unit tests, integration tests, end-to-end (E2E) testing, and validating protocol compliance.

Epic 1: Foundation – Identity, Actors, and Cryptography

Objective: The primary goal of Epic 1 is to establish the server infrastructure and implement the core Identity primitives. By the end of this phase, the system must be able to register users, generate decentralized identifiers, and expose an ActivityPub-compliant "Actor" profile that can be discovered by external Fediverse instances (e.g., Mastodon). This foundational work is critical as all subsequent social interactions rely on a valid, discoverable identity.

User Story 1.1: Decentralized Account Creation

Client Perspective: "As a new user, I want to sign up for an account that generates a secure, portable digital identity (DID) so that I can prove ownership of my username independently of the server administrators."

Context: This story sets the stage for data autonomy. Instead of just a database row, the registration process triggers the creation of a cryptographic key pair and a DID Document. The did:web method is chosen for its balance of decentralization and compatibility with web standards.[15]

- **Task 1.1.1 (BE):** Implement the did:web generation service. Upon registration, generate an RSA-2048 or Ed25519 key pair. Construct a W3C-compliant DID Document JSON containing the public key in the verificationMethod section.[4]
- **Task 1.1.2 (DO):** Configure the PostgreSQL database schema. Create a users table with columns for id (UUID), username, password_hash, and a jsonb column for the did_document. Ensure the private key is stored in a separate secrets table, encrypted at rest using pgcrypto or application-level encryption.[13]
- **Task 1.1.3 (FE):** Develop the registration form in React. Implement client-side validation for username availability and password strength. Upon successful submission, display the generated DID (e.g., did:web:instance.com:u:alice) to the user as their unique identifier.[11]

- **Task 1.1.4 (QA):** Write unit tests for the key generation module. Verify that the generated DID Document passes the official W3C DID Core specification validator, ensuring correct syntax for id, controller, and assertionMethod.[4]
- **Task 1.1.5 (BE):** Implement the /.well-known/did.json resolution endpoint (or /.well-known/did/url-encoded-id). This endpoint must serve the DID Document with the content type application/did+ld+json or application/json, allowing external verifiers to fetch the public key.[15]

User Story 1.2: ActivityPub Actor Discovery (WebFinger)

Client Perspective: "As a user, I want my profile to be searchable by friends on other platforms (like Mastodon) using a familiar handle format (e.g., @alice@my-social-network.com)."
Context: ActivityPub relies on **WebFinger** (RFC 7033) to translate human-readable handles into machine-readable Actor URLs. Without this, the platform is an island, invisible to the rest of the Fediverse.[19]

- **Task 1.2.1 (BE):** Implement the /.well-known/webfinger endpoint using the Fedify framework's built-in WebFinger tools or a custom Node.js handler. It must accept a resource query parameter (e.g., acct:alice@domain.com).[8]
- **Task 1.2.2 (BE):** Configure the WebFinger response to return a JRD (JSON Resource Descriptor) object. This object must contain a subject (the acct URI) and a link with rel="self" pointing to the ActivityPub Actor URL (e.g., https://domain.com/users/alice).[19]
- **Task 1.2.3 (QA):** Create an integration test using a tool like curl or Postman to query the WebFinger endpoint. Assert that the response headers include Content-Type: application/jrd+json and that the body contains the correct links.[2]
- **Task 1.2.4 (DO):** Configure the web server (Nginx/Railway Router) to handle the /.well-known/ path prefix correctly, ensuring it is not blocked by static asset caching rules or security filters.[17]
- **Task 1.2.5 (FE):** Create a "Profile Settings" page where the user can copy their full handle (@user@domain) to share with friends. Implement a "Copy to Clipboard" feature for better UX.[10]

User Story 1.3: The Actor Profile (JSON-LD)

Client Perspective: "As a user, I want my public profile to contain my name, bio, and avatar in a format that other apps can understand, so they can display my identity correctly."
Context: The "Actor" object is the core identity primitive in ActivityPub. It is a JSON-LD document served at the user's ID URL. It defines the user's Inbox, Outbox, and Public Key.[22]

- **Task 1.3.1 (BE):** Create the GET /users/:username endpoint. This endpoint must perform content negotiation: if the Accept header asks for application/activity+json (or ld+json), return the Actor JSON. If it asks for text/html, serve the React frontend.[1]
- **Task 1.3.2 (BE):** Construct the Actor object structure using Fedify's type-safe builders. Include mandatory fields: id, type (Person), preferredUsername, inbox, outbox,

followers, following, and publicKey.[22]

- **Task 1.3.3 (BE):** Integrate the DID Public Key into the Actor object. The publicKey.publicKeyPem field should match the key in the DID Document. This links the ActivityPub identity to the Decentralized Identity.[5]
- **Task 1.3.4 (FE):** Design the HTML fallback for the profile page. If a human visits the Actor URL in a browser, they should see a rendered React page with the user's avatar and bio, not raw JSON. This requires Server-Side Rendering (SSR) or proper routing logic.[1]
- **Task 1.3.5 (QA):** Validate the generated Actor JSON against the official ActivityStreams validator or activitypub.academy test suite to ensure strict compliance with the spec (e.g., ensuring @context is an array containing the correct string).[25]

User Story 1.4: Secure Authentication

Client Perspective: "As a registered user, I want to log in securely using my password so that I can access my account settings and post messages."
Context: While federation is decentralized, local access requires standard authentication. The project uses JWTs for stateless session management.[26]

- **Task 1.4.1 (BE):** Implement a login endpoint that verifies the password against the stored bcrypt/argon2 hash. Upon success, issue a signed JWT containing the user's UUID and DID.[26]
- **Task 1.4.2 (FE):** Build the Login UI. On successful login, store the JWT in a secure, HttpOnly cookie (if possible) or local storage, and update the global application state (Context API/Redux) to reflect the logged-in user.[27]
- **Task 1.4.3 (DO):** Set up environment variables for JWT_SECRET and COOKIE_SECRET in the Railway/Render dashboard. Ensure these secrets are never committed to version control.[28]
- **Task 1.4.4 (QA):** Perform security testing on the auth endpoints. Attempt SQL injection on the login form and verify that the API handles invalid credentials with generic error messages ("Invalid username or password") to prevent enumeration attacks.[2]
- **Task 1.4.5 (BE):** Create an authentication middleware that verifies the JWT on protected routes (e.g., POST /outbox). If the token is invalid or expired, return a 401 Unauthorized status.[26]

User Story 1.5: Profile Customization

Client Perspective: "As a user, I want to upload a profile picture and write a short bio so that I can express my personality to the network."
Context: Updating the Actor object involves changing the database and ensuring the served JSON reflects these changes immediately.

- **Task 1.5.1 (FE):** Implement an image upload component using react-dropzone. The upload should send the file to a backend endpoint which streams it to an S3-compatible object storage (e.g., AWS S3, Cloudflare R2).[29]

- **Task 1.5.2 (BE):** Create the PATCH /api/users/me endpoint. It accepts summary (bio), name (display name), and icon (avatar URL). Update the users table accordingly.[13]
- **Task 1.5.3 (BE):** Ensure the Actor JSON generation logic dynamically pulls these new fields. The icon property must be an Image object with a url and mediaType.[19]
- **Task 1.5.4 (QA):** Verify that the bio field sanitizes HTML input. Use a library like dompurify on the backend to prevent Stored XSS attacks if users attempt to inject <script> tags into their profiles.[30]
- **Task 1.5.5 (DO):** Configure the object storage bucket policies to allow public read access for the uploaded avatar images, as they need to be fetchable by remote servers.[29]

Epic 2: Content Creation and Data Autonomy

Objective: Epic 2 focuses on the "Outbox" mechanism. In ActivityPub, the Outbox is the user's publishing stream. This Epic covers the creation of posts (Notes), their storage in the database, privacy scoping, and the ability to edit or delete them. This establishes the "local" functionality of the social network before federation is introduced.

User Story 2.1: Publishing a Note

Client Perspective: "As a user, I want to write and publish a short text post (Note) so that I can share my thoughts with my followers."
Context: A "post" in ActivityPub is typically a Note object wrapped in a Create activity. The backend must construct this wrapper automatically.[1]

- **Task 2.1.1 (FE):** Develop a "Composer" component. It should include a text area, a character counter (e.g., 500 limit), and a submit button. Implement "optimistic UI" updates where the post appears in the feed immediately upon submission.[10]
- **Task 2.1.2 (BE):** Create the POST /users/:username/outbox endpoint. This endpoint accepts the raw text content. The backend must generate a unique ID (URI) for the post, typically https://domain/users/alice/statuses/<uuid>.[2]
- **Task 2.1.3 (BE):** Construct the ActivityPub object. Create a Note object with content, published, attributedTo, and to/cc fields. Then, wrap this Note in a Create activity.[31]
- **Task 2.1.4 (DO):** Design the activities table in PostgreSQL. Use a jsonb column to store the full Activity object. This schema-less approach allows for flexibility in storing different Activity types (Notes, Articles, Images) without constant migrations.[13]
- **Task 2.1.5 (QA):** Write a test case that verifies the Create activity structure. Ensure the actor field matches the attributedTo field of the Note, and that the ids are distinct and valid URIs.[22]

User Story 2.2: Viewing the Outbox (My Feed)

Client Perspective: "As a user, I want to see a chronological list of all the posts I have created so that I can review my own history."

Context: The Outbox is an OrderedCollection. Fetching it requires pagination logic to handle large histories efficiently.[1]

- **Task 2.2.1 (BE):** Implement the GET /users/:username/outbox endpoint. This should return an OrderedCollection containing the total count of items and a link to the first page of items.[1]
- **Task 2.2.2 (BE):** Implement pagination logic (OrderedCollectionPage). The first page should contain the next property linking to older posts. The database query must use LIMIT and OFFSET (or cursor-based pagination) for performance.[26]
- **Task 2.2.3 (FE):** Build the Feed component. It should fetch the Outbox JSON and render each Note item. Handle the mapping of OrderedCollection structure to a React list.[11]
- **Task 2.2.4 (FE):** Implement "Infinite Scroll" or a "Load More" button. When the user reaches the bottom of the feed, the frontend should fetch the URL provided in the next field of the current page.[10]
- **Task 2.2.5 (QA):** Performance test the Outbox query. Seed the database with 10,000 activities and verify that the response time for the first page remains under 200ms.[29]

User Story 2.3: Audience Scoping (Privacy Levels)

Client Perspective: "As a user, I want to choose whether my post is public, visible only to followers, or private, so I can control the audience of my content."
Context: ActivityPub defines privacy via the to and cc fields. There are no explicit "privacy settings" flags; instead, the addressing determines visibility.[32]

- **Task 2.3.1 (FE):** Add a privacy selector to the Composer (Public, Unlisted, Followers Only).
  - **Public:** to: [Public], cc: [Followers]
  - **Unlisted:** to: [Followers], cc: [Public]
  - **Followers Only:** to: [Followers].[33]
- **Task 2.3.2 (BE):** Implement the mapping logic. Convert the frontend selection into the appropriate ActivityStreams URIs (e.g., https://www.w3.org/ns/activitystreams#Public).[33]
- **Task 2.3.3 (BE):** Implement read-access filtering. When the Outbox is requested, check the requestor's identity. If the requestor is not in the to or cc list (and the post is not Public), exclude the activity from the response.[34]
- **Task 2.3.4 (QA):** Create a test scenario where User A posts a "Followers Only" note. User B (who does not follow A) requests A's Outbox. Assert that the sensitive note is *not* present in the returned JSON.[34]
- **Task 2.3.5 (DO):** Ensure the database index covers the visibility fields inside the JSONB column to allow efficient filtering of public vs. private content.[12]

User Story 2.4: Editing and Deleting

Client Perspective: "As a user, I want to delete a post I made by mistake, or edit a typo, so that I can maintain the quality of my content."

Context: ActivityPub handles deletion via the Delete activity and updates via the Update activity. Deletion often replaces the object with a Tombstone.[36]

- **Task 2.4.1 (FE):** Add "Edit" and "Delete" actions to the post dropdown menu. "Edit" should reload the content into the Composer.
- **Task 2.4.2 (BE):** Implement the Update handler. It should accept the new content, create an Update activity wrapping the modified Note, and update the content field in the database. The id of the Note remains the same.[31]
- **Task 2.4.3 (BE):** Implement the Delete handler. Create a Delete activity. In the database, replace the Note content with a Tombstone object (type: "Tombstone") to maintain the reference while removing the data.[26]
- **Task 2.4.4 (BE):** Ensure that fetching a Tombstoned object ID returns HTTP 410 Gone. This signals to caches and remote servers that the resource is permanently deleted.[26]
- **Task 2.4.5 (QA):** Verify that the Update activity increments the updated timestamp on the object and that the revision history is handled correctly (if implemented).[22]

User Story 2.5: Media Attachments

Client Perspective: "As a user, I want to attach an image to my post so that I can share visual experiences."
Context: The Note object has an attachment property which contains an array of Document or Image objects.[1]

- **Task 2.5.1 (BE):** Create a generic /api/media/upload endpoint. It accepts multipart/form-data, validates the file type (MIME check), and uploads it to the configured S3 storage provider.[29]
- **Task 2.5.2 (FE):** Integrate a file picker in the Composer. On file selection, upload immediately to the media endpoint and receive a URL. Display a thumbnail preview to the user.[10]
- **Task 2.5.3 (BE):** When creating the Note, map the returned media URL to the attachment field: { type: "Image", url: "...", mediaType: "image/jpeg" }.[1]
- **Task 2.5.4 (FE):** Update the Feed component to render attachments. Use a secure img tag with alt text support. Consider adding a lightbox for full-screen viewing.[11]
- **Task 2.5.5 (DO):** Implement a cleanup job. If a user uploads an image but cancels the post, the orphaned image in S3 should be deleted after 24 hours to save storage costs.

Epic 3: The Network – Federation and Interoperability

Objective: This is the core engineering challenge. Epic 3 transforms the standalone application into a networked node. It involves implementing the "Inbox" to receive messages from other servers, the "Delivery" system to send messages out, and the "WebFinger Client" to find remote users.

User Story 3.1: Remote User Lookup

Client Perspective: "As a user, I want to search for a friend on a Mastodon server (e.g., @gargron@mastodon.social) and see their profile, so I can decide whether to follow them."
Context: The system must act as a client, querying the remote server's WebFinger and then fetching the Actor object. This introduces external network dependency.[20]

- **Task 3.1.1 (FE):** Create a global search bar. Implement logic to detect if the search term matches the handle pattern (@user@domain or user@domain).
- **Task 3.1.2 (BE):** Implement the lookup service.
    1. Parse the domain.
    2. Query https://domain/.well-known/webfinger?resource=acct:user@domain.
    3. Extract the application/activity+json link.[21]
- **Task 3.1.3 (BE):** Fetch the remote Actor. Perform a GET request to the extracted link. **Crucially**, sign this GET request with the *system actor's* HTTP Signature (if required by "Secure Mode" instances) or send standard Accept headers.[21]
- **Task 3.1.4 (BE):** Cache the remote Actor. Create a remote_users table (or reuse users with a domain column). Store the Actor JSON to prevent repeated network calls. Update this cache if it's older than 24 hours.[38]
- **Task 3.1.5 (QA):** Use **Nock** or similar HTTP mocking libraries to simulate a Mastodon server's response. Test edge cases: 404 User Not Found, 500 Remote Server Error, and timeout handling.[25]

User Story 3.2: Following Remote Users

Client Perspective: "As a user, I want to click 'Follow' on a remote profile so that I can subscribe to their posts."
Context: Following requires a 3-step handshake: 1. Send Follow activity. 2. Remote server receives it. 3. Remote server sends back Accept activity. Only then is the follow confirmed.[39]

- **Task 3.2.1 (BE):** Construct the Follow activity. actor: local user URI, object: remote actor URI.
- **Task 3.2.2 (BE - Queue):** Do not send synchronously. Push the activity to the Redis delivery_queue. This decoupling is vital for reliability.[10]
- **Task 3.2.3 (BE - Worker):** Create a background worker (BullMQ) that processes the queue. It resolves the remote actor's inbox, signs the request (HTTP Signature), and POSTs the Follow activity.[40]
- **Task 3.2.4 (BE):** Record the relationship in a follows table with status: pending.
- **Task 3.2.5 (FE):** Update the UI button to show "Requested" or "Pending" state until the Accept signal is processed (Epic 3.3).[11]

User Story 3.3: Processing the Inbox (Incoming Federation)

Client Perspective: "As a user, I want to receive posts from people I follow on other servers in my home feed."
Context: The Inbox is the listening port of the server. It receives Create, Update, Delete,

Follow, Accept, Undo activities from the world.[22]

- **Task 3.3.1 (BE):** Implement POST /users/:username/inbox. It must accept JSON-LD.
- **Task 3.3.2 (BE):** Implement HTTP Signature Verification (see Epic 5 for full security, but basic structural validation starts here).
- **Task 3.3.3 (BE - Dispatcher):** Create a handler for the Create activity.
    1. Verify the actor matches the attributedTo of the object.
    2. Check if the local user actually follows the actor (spam prevention).
    3. Save the Note to the activities table.[22]
- **Task 3.3.4 (BE - Dispatcher):** Create a handler for the Accept activity. When received, find the corresponding "pending" follow request in the database and upgrade it to "accepted".[39]
- **Task 3.3.5 (QA):** Use the **Fedify CLI** (fedify inbox) or activitypub.academy to send a mock Create activity to the local server. Verify it appears in the database.[25]

User Story 3.4: Shared Inbox Optimization

Client Perspective: "As a user, I want the system to handle popular threads without crashing, ensuring reliable service."
Context: Without a Shared Inbox, if you follow 100 people on mastodon.social, that server sends 100 identical messages to your server. A Shared Inbox receives *one* message, and the local server distributes it internally.[23]

- **Task 3.4.1 (BE):** Implement the POST /inbox (Global Shared Inbox) endpoint.
- **Task 3.4.2 (BE):** Update the Actor generation to include the endpoints: { sharedInbox: "https://domain/inbox" } property.[23]
- **Task 3.4.3 (BE):** Implement "Fan-out" logic. When a message arrives at the Shared Inbox:
    1. Identify the sender.
    2. Query the follows table to find *all* local users following this sender.
    3. Insert the activity into the feed/inbox of those specific users.[40]
- **Task 3.4.4 (DO):** Monitor the Shared Inbox queue depth. If fan-out is slow, increase the number of Redis worker processes.[29]
- **Task 3.4.5 (QA):** Load test the fan-out logic. Simulate a payload with 500 local recipients and measure the time to consistency (TTC) for all feeds.[29]

User Story 3.5: The Aggregated Timeline

Client Perspective: "As a user, I want to see a mixed stream of my own posts and posts from people I follow, sorted by time."
Context: This requires a complex SQL query merging local activities and the cached remote activities ingested via the Inbox.

- **Task 3.5.1 (BE):** Optimize the SQL query. SELECT * FROM activities WHERE actor_id IN (list_of_followed_ids) ORDER BY published DESC. Use database indexes on published and actor_id.[13]
- **Task 3.5.2 (BE):** Handle "Announce" (Boost) activities in the timeline. If User A boosts a

post by User B, show User B's post with a "Boosted by A" header.[42]

- **Task 3.5.3 (FE):** Update the Feed UI to distinctively mark remote posts (e.g., show the full handle).
- **Task 3.5.4 (BE):** Implement "Context Fetching". If a remote post is a reply to a post the server *doesn't* have, the backend should attempt to fetch the parent post URL to fill the gap (backfilling).[43]
- **Task 3.5.5 (QA):** verify that the timeline strictly respects the published date, handling time zone differences correctly (store everything as UTC).[44]

Epic 4: Engagement and Interaction

Objective: A social network is defined by interaction. This Epic adds depth: threading (replies), endorsements (likes), and re-sharing (boosts). It transforms the feed from a list of statements into a conversation.

User Story 4.1: Threaded Conversations

Client Perspective: "As a user, I want to reply to a post and see the conversation organized in a thread, so I can follow the discussion."
Context: Threading relies on the inReplyTo property. A reply is just a Note that points to another Note.[30]

- **Task 4.1.1 (FE):** Update the Composer to support "Reply Mode". When replying, the UI should show the parent post and pre-fill the privacy scope to match the parent.
- **Task 4.1.2 (BE):** When creating a reply, set the inReplyTo field to the parent ID. Add the parent author to the to field (so they get notified).[31]
- **Task 4.1.3 (FE):** Build a "Thread View" component. This is a recursive UI structure. It fetches the focus post, its ancestors (parents), and its descendants (children).
- **Task 4.1.4 (BE):** Implement GET /api/statuses/:id/context. This endpoint queries the database for all posts where inReplyTo equals the current ID (children) and the post referenced by the current inReplyTo (parent).[43]
- **Task 4.1.5 (QA):** Test deep nesting. Verify that the UI handles 10+ levels of replies gracefully without breaking the layout (e.g., indentation limits).[10]

User Story 4.2: Likes and Favorites

Client Perspective: "As a user, I want to 'Like' a post to show my appreciation to the author."
Context: The Like activity. Unlike Notes, Likes are not usually "content" but "actions" on content.[22]

- **Task 4.2.1 (FE):** Add a "Like" button (Heart). Implement "optimistic" state—turn red immediately on click.
- **Task 4.2.2 (BE):** Generate the Like activity. object: the post URI. Deliver this activity to the post author's inbox.[31]
- **Task 4.2.3 (BE):** Increment a likes_count counter on the local post record. This is a

denormalization for performance.

- **Task 4.2.4 (BE):** Handle Undo Like. If the user clicks the button again, send an Undo activity wrapping the original Like. Decrement the counter.[36]
- **Task 4.2.5 (FE):** Display the like count. If the post is remote, this count might only reflect *local* likes unless the remote server's count is fetched periodically (a known ActivityPub quirk).[45]

User Story 4.3: Boosting (Re-sharing)

Client Perspective: "As a user, I want to 'Boost' a post to share it with my own followers."
Context: The Announce activity. This is the primary mechanism for virality in the Fediverse.[42]

- **Task 4.3.1 (BE):** Generate the Announce activity. object: the original post URI. to: the user's Followers collection.[1]
- **Task 4.3.2 (BE):** Deliver the Announce to all followers. Crucially, if the original post is remote, this notifies the original author that their post was shared (if they are in the cc).
- **Task 4.3.3 (FE):** Filter the timeline. If the user boosts a post, do not show the original post *and* the boost adjacent to each other if possible (deduplication logic).
- **Task 4.3.4 (QA):** Verify privacy constraints. Ensure the UI prevents boosting "Followers Only" posts, as this would violate the original author's intent (though the protocol allows it, good clients prevent it).[34]
- **Task 4.3.5 (BE):** Handle Undo Announce (Unboost). Remove the Announce activity from the database and send the Undo to followers.

User Story 4.4: Notifications System

Client Perspective: "As a user, I want to get a notification when someone follows me, replies to me, or likes my post."
Context: Aggregating inbound activities that are "directed" at the user.

- **Task 4.4.1 (DO):** Create a notifications table: id, recipient_id, actor_id, type (follow, like, reply, mention), object_id, read_boolean.[13]
- **Task 4.4.2 (BE):** Update the Inbox Dispatcher. When a relevant activity arrives, insert a row into the notifications table.
- **Task 4.4.3 (FE):** Create a Notifications page. Group notifications? (e.g., "3 people liked your post"). Display the actor's avatar and the action type.
- **Task 4.4.4 (BE):** Implement a "Mark as Read" endpoint.
- **Task 4.4.5 (FE):** Add a badge to the navigation bar showing the count of unread notifications. Poll the endpoint every 60 seconds or use a WebSocket for real-time updates.[46]

User Story 4.5: Custom Emojis

Client Perspective: "As a user, I want to use custom emojis in my posts to express myself better, compatible with other instances."
Context: ActivityPub custom emojis are defined in the tag property of the Object.[30]

- **Task 4.5.1 (BE):** Create a table for custom_emojis (shortcode, image_url).

- **Task 4.5.2 (FE):** Integrate an emoji picker that mixes standard unicode emojis with the custom ones fetched from the backend.
- **Task 4.5.3 (BE):** When processing outgoing posts, scan the text for shortcodes (e.g., :blobcat:). If found, append the emoji metadata to the tag array in the ActivityPub object: { type: "Emoji", name: ":blobcat:", icon: { url: "..." } }.[30]
- **Task 4.5.4 (FE):** When rendering posts, verify the tag array. If an emoji is present, replace the shortcode in the text with an inline <img> tag using the provided URL.
- **Task 4.5.5 (QA):** Test accessibility. Ensure that the custom emoji img tag includes alt=":shortcode:" so screen readers can describe it.[47]

Epic 5: Trust, Safety, and Production Governance

Objective: The final Epic addresses the critical "Trust and Safety" layer. An open federation allows bad actors. This phase implements the cryptographic proof required for secure federation (HTTP Signatures) and the moderation tools required to run a safe community. It concludes with production deployment.

User Story 5.1: HTTP Signatures (Security)

Client Perspective: "As a user, I want to be sure that no one can impersonate me or fake messages from me."
Context: ActivityPub uses **HTTP Signatures** (Draft-Cavage-12) to authenticate server-to-server requests. This is mandatory for federation with Mastodon.[2]

- **Task 5.1.1 (BE):** Implement the Signing Middleware. For every outgoing request, create a string comprising the (request-target), host, date, and digest. Sign this string using the user's private key (from Epic 1).[48]
- **Task 5.1.2 (BE):** Attach the Signature header to the request. Include the keyId which points to the public key in the user's Actor profile.[49]
- **Task 5.1.3 (BE):** Implement the Verification Middleware. For every incoming request to the Inbox, parse the signature. Fetch the public key from the keyId URL (caching it). Verify the signature matches the body.[48]
- **Task 5.1.4 (BE):** Implement "Double-Knocking" (optional but good). If the signature is valid, perform a reverse DNS lookup or check that the actor resides on the domain claiming to send the message.
- **Task 5.1.5 (QA):** Use the Fedify testing suite to send requests with invalid signatures, expired timestamps, or tampered bodies. Ensure the server returns 401 Unauthorized.[25]

User Story 5.2: Blocking and Muting

Client Perspective: "As a user, I want to block abusive users or entire servers so I don't see their content."
Context: The Block activity. Note that blocking a *domain* is a server-level action, while blocking a *user* is an account-level action.[30]

- **Task 5.2.1 (FE):** Add "Block" options to profiles and posts.
- **Task 5.2.2 (BE):** Generate a Block activity. Store the block in a blocks table. **Do not** send the Block activity to the blocked user (standard privacy practice to avoid retaliation) unless specifically desired.[30]
- **Task 5.2.3 (BE):** Update the Feed Query. Filter out any activities where the actor_id is in the user's block list.
- **Task 5.2.4 (BE):** Implement Domain Blocks. Create a domain_blocks table. In the Inbox, check the origin domain of every incoming activity. If blocked, drop the packet immediately.[30]
- **Task 5.2.5 (QA):** Verify that if User A blocks User B, User A no longer receives notifications from B, and B cannot see A's posts (if A's server correctly filters the Outbox).[30]

User Story 5.3: Reporting (Flagging)

Client Perspective: "As a user, I want to report content that violates the server rules to the administrators."
Context: The Flag activity is used to report content. It is usually sent to the instance admin.[30]

- **Task 5.3.1 (FE):** Create a Report Modal. Allow the user to select the specific post and write a reason for the report.
- **Task 5.3.2 (BE):** Generate a Flag activity. object: the post(s), content: the reason. Deliver this to the **local** system admin's Inbox.
- **Task 5.3.3 (BE):** Build an Admin Dashboard. A restricted route /admin/reports that lists incoming Flags.
- **Task 5.3.4 (BE):** Implement Admin Actions: "Delete Local Copy," "Suspend User," "Silence Domain."
- **Task 5.3.5 (QA):** Test the workflow: User reports -> Flag arrives in DB -> Admin views Flag -> Admin hides post.

User Story 5.4: Server Metadata (NodeInfo)

Client Perspective: "As a user, I want to see statistics about the server (user count, software version) to know if it's active."
Context: NodeInfo is a standardized protocol for server metadata. It is how sites like fedidb.org crawl the network.[41]

- **Task 5.4.1 (BE):** Implement /.well-known/nodeinfo. This returns a JSON pointing to the actual metadata endpoint (e.g., /nodeinfo/2.0).
- **Task 5.4.2 (BE):** Implement /nodeinfo/2.0. Return JSON containing: version (2.0), software (name: "student-project", version: "1.0"), protocols (["activitypub"]), usage (users: { total: X, activeMonth: Y }).[41]
- **Task 5.4.3 (BE):** Cache these statistics. Do not calculate COUNT(*) on the users table for every request. Update stats hourly.
- **Task 5.4.4 (FE):** Create an "About" page that displays these stats nicely, along with the admin's contact email and the privacy policy.

- **Task 5.4.5 (QA):** Validate the endpoint against the NodeInfo schema schema.org.

User Story 5.5: Production Deployment

Client Perspective: "As a user, I want to access the platform at a real URL (https://social.example.com) that is secure and always online."
Context: Moving from localhost to the live web via Railway/Render.[17]

- **Task 5.5.1 (DO):** Purchase a domain name (or use a free subdomain). Configure DNS records (A, CNAME) to point to the Railway/Render load balancer.
- **Task 5.5.2 (DO):** Configure SSL/TLS. Railway handles this automatically, but ensure did:web resolution matches the certificate domain exactly.[15]
- **Task 5.5.3 (DO):** Set up the CI/CD pipeline in GitHub Actions.
  - Step 1: Run Unit Tests.
  - Step 2: Build Docker Container.
  - Step 3: Deploy to Staging.
  - Step 4: (Manual Approval) Deploy to Production.[17]
- **Task 5.5.4 (BE):** Configure Database Migrations. Ensure that when the new code deploys, any schema changes (e.g., adding domain_blocks table) run automatically.
- **Task 5.5.5 (QA):** Post-Deployment Smoke Test. Run a script that registers a new user, posts a note, and queries the WebFinger endpoint on the production URL to confirm system integrity.

---

3. Risk Management and Future Scope

3.1 Technical Risks

- **DID/ActivityPub Mismatch:** ActivityPub assumes HTTPS-based identity. Bridging this with DIDs is complex. *Mitigation:* Use did:web which maps inherently to HTTPS, ensuring compatibility without requiring custom resolvers on remote instances.[15]
- **Signature Verification Complexity:** Implementing HTTP Signatures correctly is the most common failure point. *Mitigation:* Use the **Fedify** library which abstracts the cryptographic primitives, rather than implementing raw RSA signing manually.[8]
- **Spam/Abuse:** Opening a federation port invites spam. *Mitigation:* Implement "Authorized Fetch" (Secure Mode) where the server ignores any request that isn't signed by a known, valid actor.[35]

3.2 Future Scope

While this semester project focuses on the core federation, future iterations could explore:

- **Account Migration:** Using the Move activity to port the DID-based identity to a new server.[30]
- **End-to-End Encryption (E2EE):** Using the DID public keys to encrypt Direct Messages, ensuring only the recipient can read them, bypassing the server admin.[51]

- **BlueSky Bridge:** Implementing the AT Protocol bridge to allow interaction with BlueSky users.[52]

Conclusion

This architectural blueprint provides a rigorous, step-by-step path for a student team to build a production-grade Federated Social Network. By adhering to the W3C standards for ActivityPub and Decentralized Identifiers, the project not only teaches full-stack engineering but also immerses students in the ethical and technical complexities of the decentralized web. The use of modern, type-safe tooling (Fedify, TypeScript) ensures the project is manageable within a semester while producing a portfolio-worthy artifact that stands at the cutting edge of social media technology.

Works cited

1. A Guide to Implementing ActivityPub in a Static Site (or Any Website) — Part 4 - Medium, accessed on December 22, 2025, https://medium.com/@maho.pacheco.m/a-guide-to-implementing-activitypub-in-a-static-site-or-any-website-part-4-361185072bba
2. Fediverse: Exploring the Federated Web & A Beginner's Guide to ..., accessed on December 22, 2025, https://dev.to/austinwdigital/fediverse-exploring-the-federated-web-a-beginners-guide-to-activitypub-development-2hc8
3. How does Mastodon work? | SURF.nl, accessed on December 22, 2025, https://www.surf.nl/en/how-does-mastodon-work
4. Decentralized Identifiers (DIDs) v1.0 - W3C, accessed on December 22, 2025, https://www.w3.org/TR/did-1.0/
5. Decentralized Identifiers (DIDs) v1.1 - W3C, accessed on December 22, 2025, https://www.w3.org/TR/did-1.1/
6. Creating your own federated microblog - Fedify, accessed on December 22, 2025, https://fedify.dev/tutorial/microblog
7. Learning the basics of Fedify, accessed on December 22, 2025, https://fedify.dev/tutorial/basics
8. Fedify - JSR, accessed on December 22, 2025, https://jsr.io/@fedify/fedify
9. fedify-dev/fedify: ActivityPub server framework in TypeScript - GitHub, accessed on December 22, 2025, https://github.com/fedify-dev/fedify
10. Mastodon Tech Stack - Himalayas.app, accessed on December 22, 2025, https://himalayas.app/companies/mastodon/tech-stack
11. React – A JavaScript library for building user interfaces, accessed on December 22, 2025, https://legacy.reactjs.org/
12. Documentation: 18: Chapter 35. The Information Schema - PostgreSQL, accessed on December 22, 2025, https://www.postgresql.org/docs/current/information-schema.html
13. ActivityPub in Pleroma - Lainblog, accessed on December 22, 2025, https://blog.soykaf.com/post/activity-pub-in-pleroma/

14. Comparing Decentralized Identifiers(DID) Methods - DEV Community, accessed on December 22, 2025, https://dev.to/lymah/comparing-decentralized-identifiersdid-methods-el
15. Announcing Public Review of the did:webs Method Specification - Trust Over IP, accessed on December 22, 2025, https://trustoverip.org/news/2023/12/15/announcing-public-review-of-the-didwebs-method-specification/
16. Deploy for Free – Render Docs, accessed on December 22, 2025, https://render.com/docs/free
17. Deploy a Fastify App - Railway Docs, accessed on December 22, 2025, https://docs.railway.com/guides/fastify
18. Documentation: 18: 5.10. Schemas - PostgreSQL, accessed on December 22, 2025, https://www.postgresql.org/docs/current/ddl-schemas.html
19. A Guide to Implementing ActivityPub in a Static Site (or Any Website) - Part 3, accessed on December 22, 2025, https://maho.dev/2024/02/a-guide-to-implementing-activitypub-in-a-static-site-or-any-website-part-3/
20. ActivityPub - Indie Microblogging, accessed on December 22, 2025, https://book.micro.blog/activitypub/
21. A bare-minimum ActivityPub server from scratch - Гришкин блог, accessed on December 22, 2025, https://grishka.me/blog/activitypub-from-scratch/
22. ActivityPub - W3C, accessed on December 22, 2025, https://www.w3.org/TR/activitypub/
23. Actor dispatcher - Fedify, accessed on December 22, 2025, https://fedify.dev/manual/actor
24. ActivityPub Over ATProto - Robin Berjon, accessed on December 22, 2025, https://berjon.com/ap-at/
25. Testing | Fedify, accessed on December 22, 2025, https://fedify.dev/manual/test
26. Modular ActivityPub implementation as Express JS middleware to easily add decentralization and federation to Node apps - GitHub, accessed on December 22, 2025, https://github.com/immers-space/activitypub-express
27. React, accessed on December 22, 2025, https://react.dev/
28. How to Deploy a Node.js App on Railway in Under 10 Minutes - DEV Community, accessed on December 22, 2025, https://dev.to/arunangshu_das/how-to-deploy-a-nodejs-app-on-railway-in-under-10-minutes-1fem
29. Deploying a Mastodon/ActivityPub Server: Hardware & Setup Guide - WeHaveServers.com, accessed on December 22, 2025, https://wehaveservers.com/blog/dev-use-cases/deploying-a-mastodon-activitypub-server-hardware-setup-guide/
30. ActivityPub - Mastodon documentation, accessed on December 22, 2025, https://docs.joinmastodon.org/spec/activitypub/
31. ActivityPub - W3C on GitHub, accessed on December 22, 2025, https://w3c.github.io/activitypub/
32. ActivityPub/Primer/Addressing - W3C Wiki, accessed on December 22, 2025,

https://www.w3.org/wiki/ActivityPub/Primer/Addressing
33. Visibility ↔ To/Cc mapping - Mastodon - SocialHub, accessed on December 22, 2025, https://socialhub.activitypub.rocks/t/visibility-to-cc-mapping/284
34. Public and Private Visibility - NodeBB Documentation, accessed on December 22, 2025, https://docs.nodebb.org/activitypub/visibility/
35. ActivityPub/Primer/Authentication Authorization - W3C Wiki, accessed on December 22, 2025, https://www.w3.org/wiki/ActivityPub/Primer/Authentication_Authorization
36. ActivityPub, accessed on December 22, 2025, http://rhiaro.github.io/activitypub/
37. Understanding ActivityPub - Part 1: Protocol Fundamentals - Sebastian Jambor's blog, accessed on December 22, 2025, https://seb.jambor.dev/posts/understanding-activitypub/
38. Activitypub & Mastodon, how to get an Actor? - Stack Overflow, accessed on December 22, 2025, https://stackoverflow.com/questions/75025657/activitypub-mastodon-how-to-get-an-actor
39. ActivityPub/Primer/Follow activity - W3C Wiki, accessed on December 22, 2025, https://www.w3.org/wiki/ActivityPub/Primer/Follow_activity
40. Sending activities - Fedify, accessed on December 22, 2025, https://fedify.dev/manual/send
41. Hidden Gems of the Fedify CLI: Tips & Tricks You Might Have Missed - DEV Community, accessed on December 22, 2025, https://dev.to/hongminhee/hidden-gems-of-the-fedify-cli-tips-tricks-you-might-have-missed-54af
42. ActivityPub | PeerTube documentation - JoinPeerTube, accessed on December 22, 2025, https://docs.joinpeertube.org/api/activitypub
43. A Guide to Implementing ActivityPub in a Static Site (or Any Website) - Part 8, accessed on December 22, 2025, https://maho.dev/2025/01/a-guide-to-implementing-activitypub-in-a-static-site-or-any-website-part-8/
44. Mastodon (social network) - Wikipedia, accessed on December 22, 2025, https://en.wikipedia.org/wiki/Mastodon_(social_network)
45. ActivityPub - Wikipedia, accessed on December 22, 2025, https://en.wikipedia.org/wiki/ActivityPub
46. Best & free way to deploy a Node.js backend, just for development - Reddit, accessed on December 22, 2025, https://www.reddit.com/r/node/comments/1mi9csp/best_free_way_to_deploy_a_nodejs_backend_just_for/
47. The Glitch Community's Guide to Building the Fediverse, accessed on December 22, 2025, https://blog.glitch.com/post/glitch-communitys-guide-to-building-the-fediverse
48. ActivityPub and HTTP Signatures, accessed on December 22, 2025, https://swicg.github.io/activitypub-http-signature/
49. misskey-dev/node-http-message-signatures: An JavaScript (Node.js and browsers) implementation for HTTP Message Signatures (RFC 9421) - GitHub,

accessed on December 22, 2025, https://github.com/misskey-dev/node-http-message-signatures
50. Fedify, accessed on December 22, 2025, https://fedify.dev/
51. Architectural variations | activitypub-e2ee, accessed on December 22, 2025, https://swicg.github.io/activitypub-e2ee/architectural-variations
52. Should ActivityPub and ATProtocol be Potentially Merged into a Single Protocol? - Reddit, accessed on December 22, 2025, https://www.reddit.com/r/fediverse/comments/1nxzx8r/should_activitypub_and_atprotocol_be_potentially/
53. Interoperability with Mastodon/ActivityPub · bluesky-social atproto · Discussion #1716 - GitHub, accessed on December 22, 2025, https://github.com/bluesky-social/atproto/discussions/1716